

先進計算機構成論 03

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

shioya@ci.i.u-tokyo.ac.jp

今日の内容

- 回路の消費電力の補足
 1. アーキテクチャの違いによる消費電力の違い
 2. FPGA による回路
 3. (余談) ムーアの法則と周波数
- パイプライン化の基本

回路の消費電力の補足

CPU やその他回路の消費電力について

■ いくつか補足

1. **アーキテクチャの違いによる消費電力の違い**
2. FPGA による回路
3. (余談) ムーアの法則と周波数

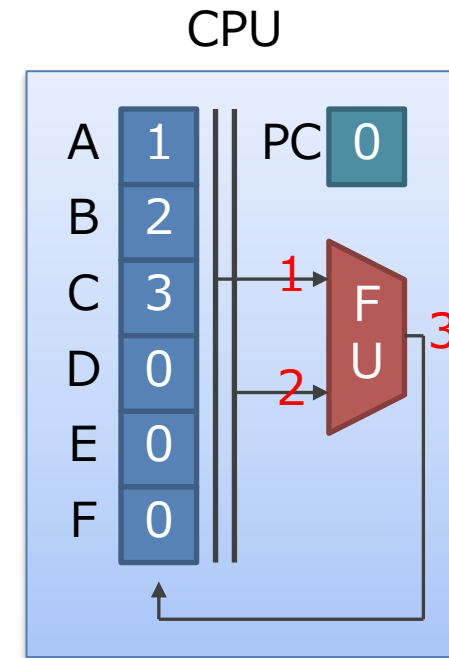
回路の遅延と消費エネルギー

- 回路の消費エネルギー：
 - 主にコンデンサへの充放電で消費
- おおざっぱには、トランジスタ数に比例すると考えて良い
 - トランジスタ数が増えると、コンデンサが増える
 - トランジスタ数 $\propto$ 回路面積

命令を処理するのに必要な回路

■ 内訳：

- 命令の読み出し
- デコード
- レジスタ読み書き
- メモリの読み書き
- （命令のスケジューリングや投機関係など
- 演算



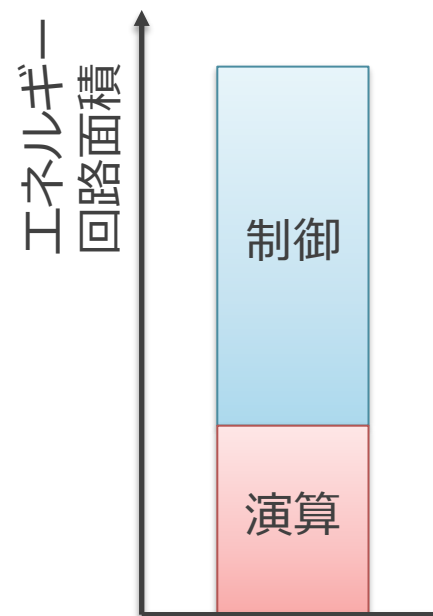
回路は命令制御と演算に大きく分けられる

■ 命令制御：アーキテクチャによって大きく異なる

- 命令の読み出し
- デコード
- レジスタ読み書き
- （命令のスケジューリングや投機関係など

■ 演算：基本的に同じ

- 論理演算，算術演算，浮動小数点演算
- これら演算単体は，だいたいどのアーキでも同じことをする



⇒ 命令制御部分をどれだけケチれるかが，アーキの効率を大きく決める

いろいろな回路の規模

- 1bit NAND 演算器 : 4 トランジスタ
(小さすぎて見えない)
- 64bit 整数加算器 : 4k トランジスタ

- MIPS R3000 プロセッサ : 115k トランジスタ

- 64bit 浮動小数点 乗算+加算器 : 68k トランジスタ

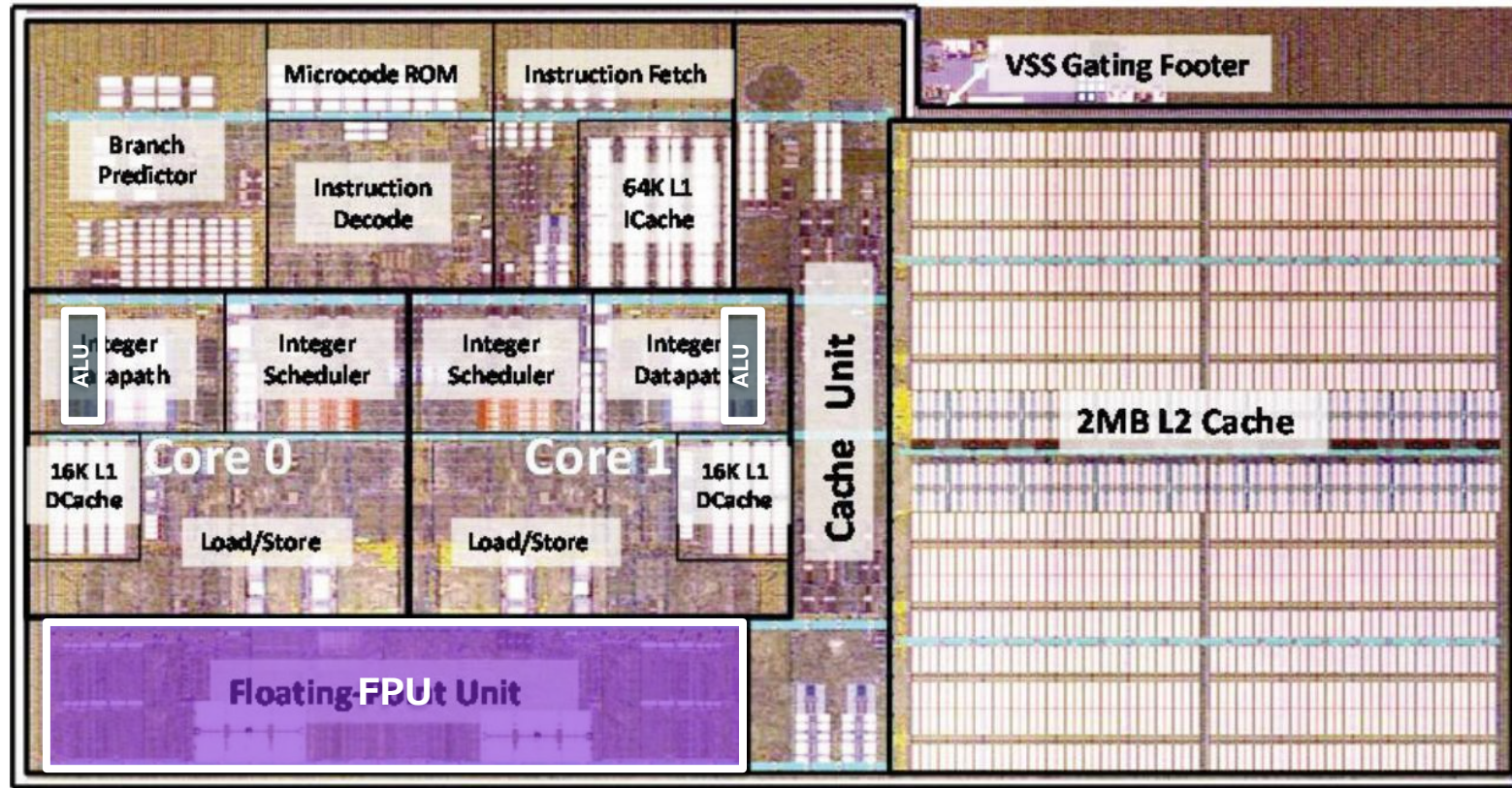

Ryota Shioya, Masahiro Goshima, and Hideki Ando: A Front-end Execution Architecture for High Energy Efficiency, MICRO 2014.

SOHN, Jongwook, et al.: Enhanced Floating-Point Multiply-Add with Full Denormal Support, ARITH 2023

回路規模の例からわかること

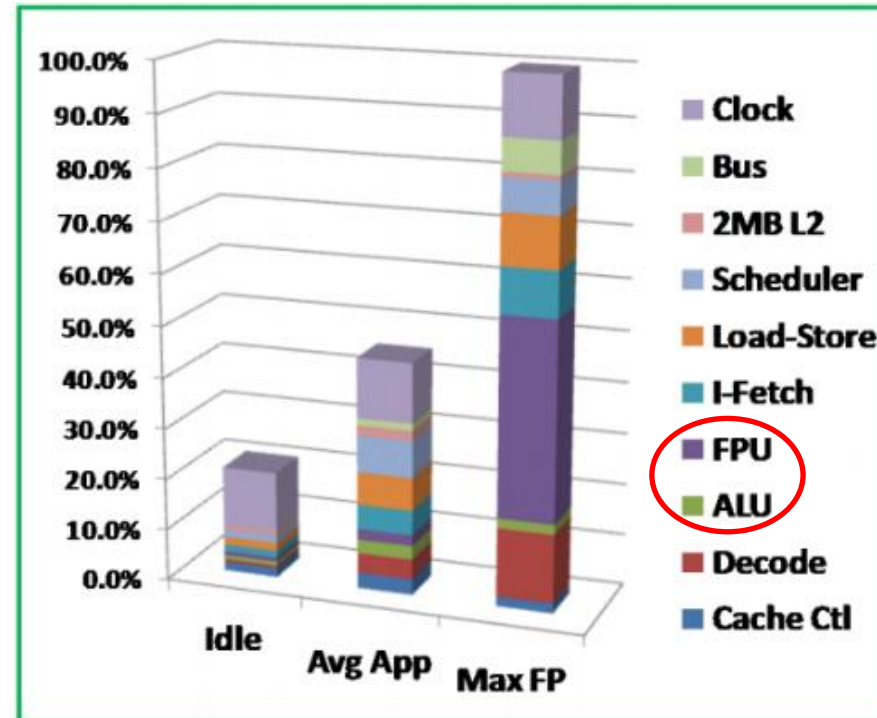
- MIPS プロセッサ全体に対する相対的な大きさで考える：
 - 1ビット論理演算：
 - 命令制御がほぼ全てを占める
 - 64 ビット加算
 - 命令制御が大半を占める
⇒ 専用回路にすると一桁ぐらい効率が上がりうる
 - FP 演算
 - 演算器の方やや小さい
⇒ 専用回路にすると2倍ぐらいは効率が良くなるかも
- 消費エネルギーも、これに準じた大きさとなる

AMD Bulldozer のチップ写真



- Tim Fischer¹, Srikanth Arekapudi², Eric Busta¹, Carl Dietz³, Michael Golden², Scott Hilker², Aaron Horiuchi¹, Kevin A. Hurd¹, Dave Johnson¹, Hugh McIntyre², Samuel Naffziger¹, James Vinh², Jonathan White⁴, Kathryn Wilcox, Design Solutions for the Bulldozer 32nm SOI 2-Core Processor Module in an 8-Core CPU, ISSCC 2011 より

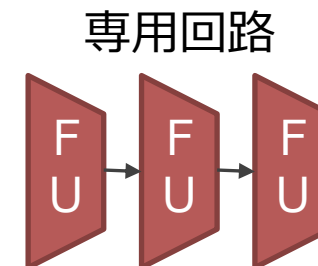
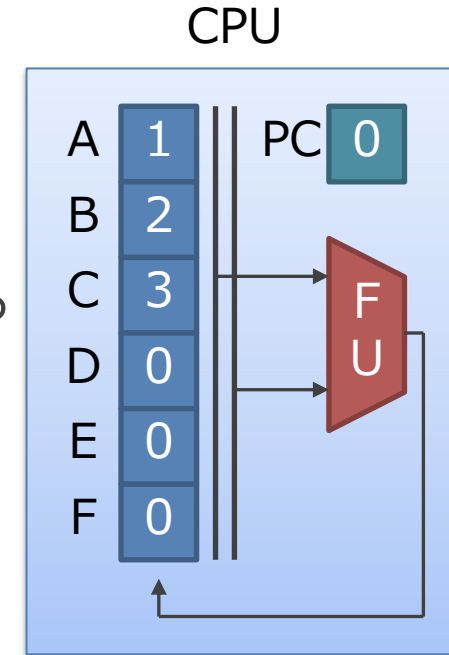
AMD Steamroller の消費電力



- ALU : 整数演算器
 - 全体からみると, 大きな割合を占めていない
- FPU : FP 演算器
 - 稼働時 (Max FP) には, かなり大きな割合を占める

エネルギーは命令制御と演算の比率によって決まる

- CPU：演算以外の制御部分が多い
 - 基本 1 つの命令が 1 つのデータを操作
 - 高速化のため命令の実行順を入れ替える等もする
- GPU：制御部分が相対的に小さい
 - 1 つの命令で多数のデータを操作
 - 命令フェッチ/デコードなどに必要な分が減る
- 専用回路：制御部分やレジスタがない
 - そもそも命令で処理しない
 - 演算器のみが繋がったような構造に流し込む
 - （演算器の幅自体をさらに最適化することもある



使いやすさは、おおむね制御部分の大きさに比例

- CPU：制御部分大きい
 - ほっといても、ハードが（ある程度）勝手に並列実行してくれる
 - プログラマが一番楽
- GPU：制御部分が小さい
 - 単一の命令で複数のデータを操作
 - 規則正しくデータが並んでいるようにお膳立てしないと性能がでない
- 専用回路：制御部分が（あまり）ない
 - そもそもプログラムを実行できない
 - 目的ごとに回路の設計からしないといけない

牧野先生のアーキテクチャの「効率」の定義

- Principles of High-Performance Processor Design
For High Performance Computing, Deep Neural Networks and Data Science
Junichiro Makino
<https://link.springer.com/book/10.1007/978-3-030-76871-3>
- あるアーキテクチャにおいて、あるアプリケーションを実行する場合に、
 - FP 演算そのものに必要な最小のエネルギー： $P_{\min}$
 - 実際に消費されたエネルギー： P_{actual}
 - 効率： $P_{\min}/P_{actual}$
- 演算以外の余計なことをどれだけ減らすかが効率を決めるという点で、おおよそ同じ事を言っている

CPU やその他回路の消費電力について

■ 消費電力について

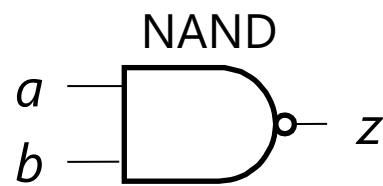
1. クロックの消費電力
2. アーキテクチャの違いによる消費電力の違い
3. **FPGA による回路**

FPGA の場合

- FPGA : Field-Programmable Gate Array
 - 中身を書き換えることのできる回路
- FPGA で専用回路を作れば, いいことばかり?
 - 設計の敷居が下がりつつ, 電力効率もよくなる?
 - CPU で実行されるプログラムの処理を専用回路に置き換える
- そんなに単純な話ではない
 1. FPGA の仕組み
 2. FPGA でうまく行く場合と行かない場合

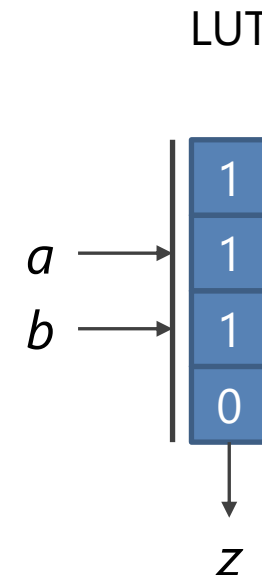
FPGA の仕組み

- 書き換え可能なテーブルにより, 回路を実現
 - LUT : Look up Table
 - 真理値表そのものを保持するテーブル
 - 事前にこれを所望の回路の真理値表に設定しておく
 - 入力をインデックスとしてテーブルにアクセスし, 出力
 - 下の NAND の場合, a と b の 2ビットのインデックス



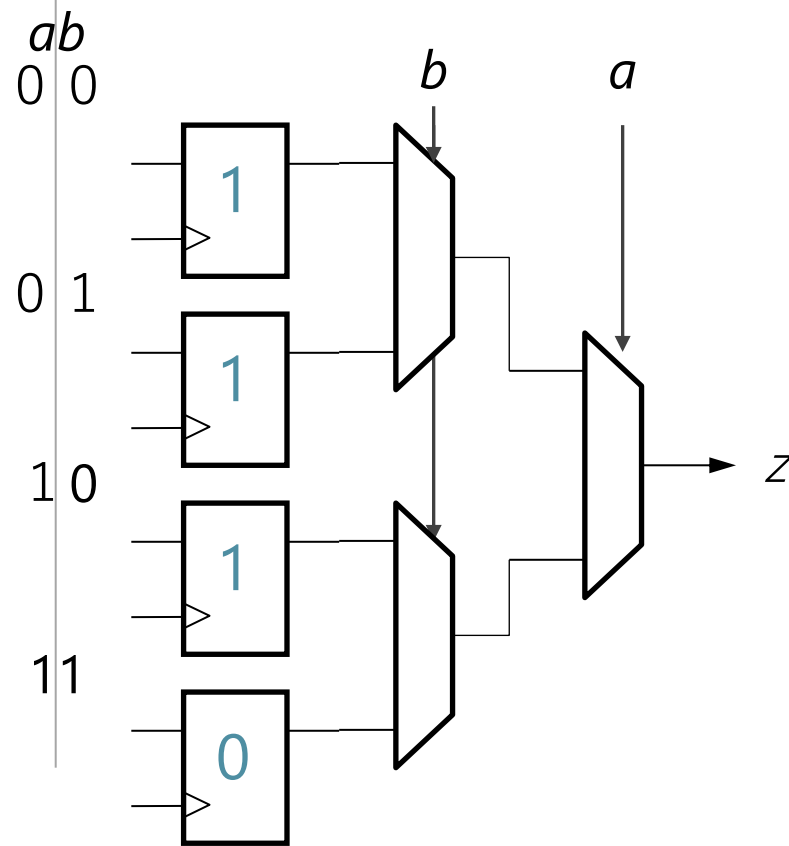
真理値表

a	b	z
0	0	1
0	1	1
1	0	1
1	1	0



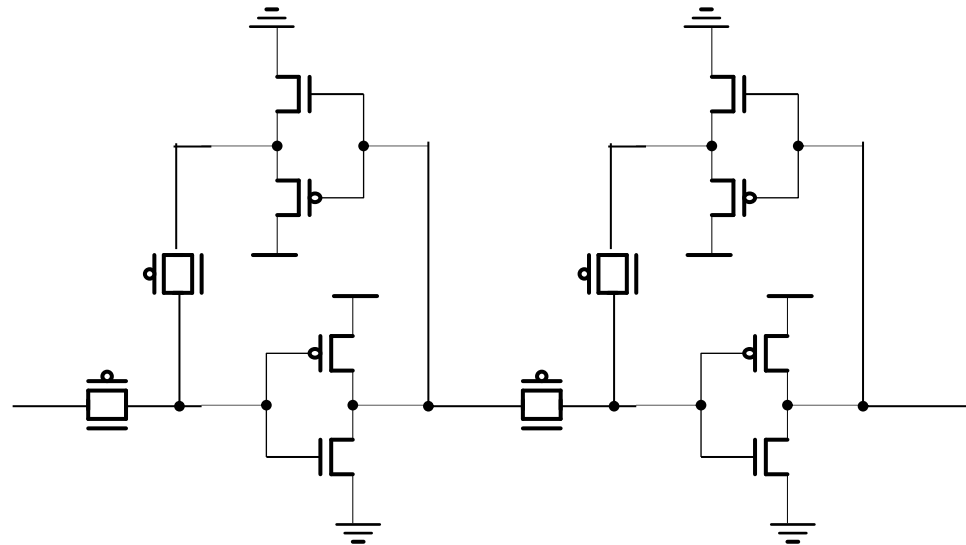
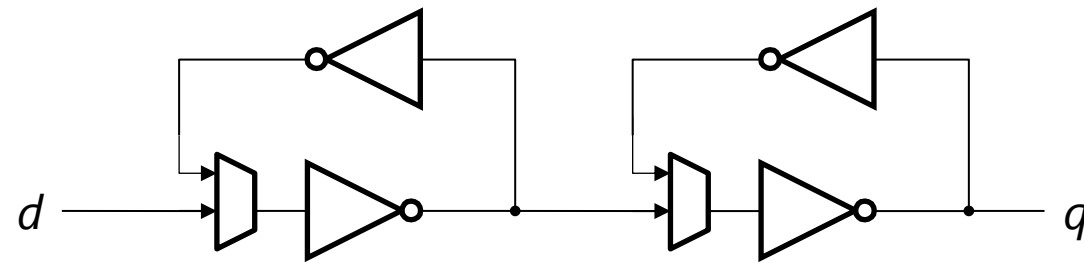
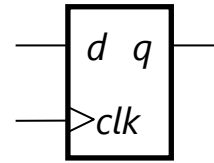
LUT の回路量の見積もり

NAND		
a	b	z
0	0	1
0	1	1
1	0	1
1	1	0

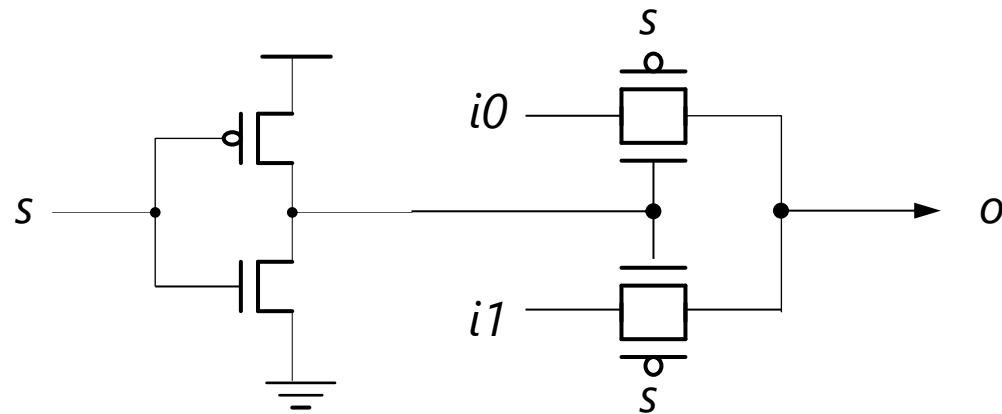
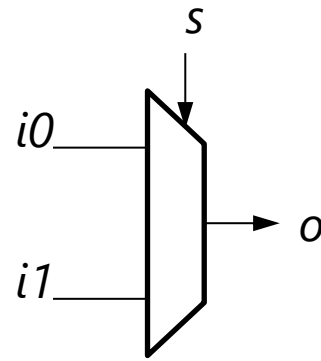


- 4 エントリの LUT を D-FF で構成してみる
 - 中身を憶える 4つの D-FF
 - 場所を指定して選択する2段のマルチプレクサ

D-FF : トランジスタ 16個



マルチプレクサ：トランジスタ 6個

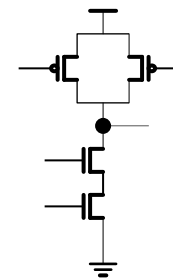
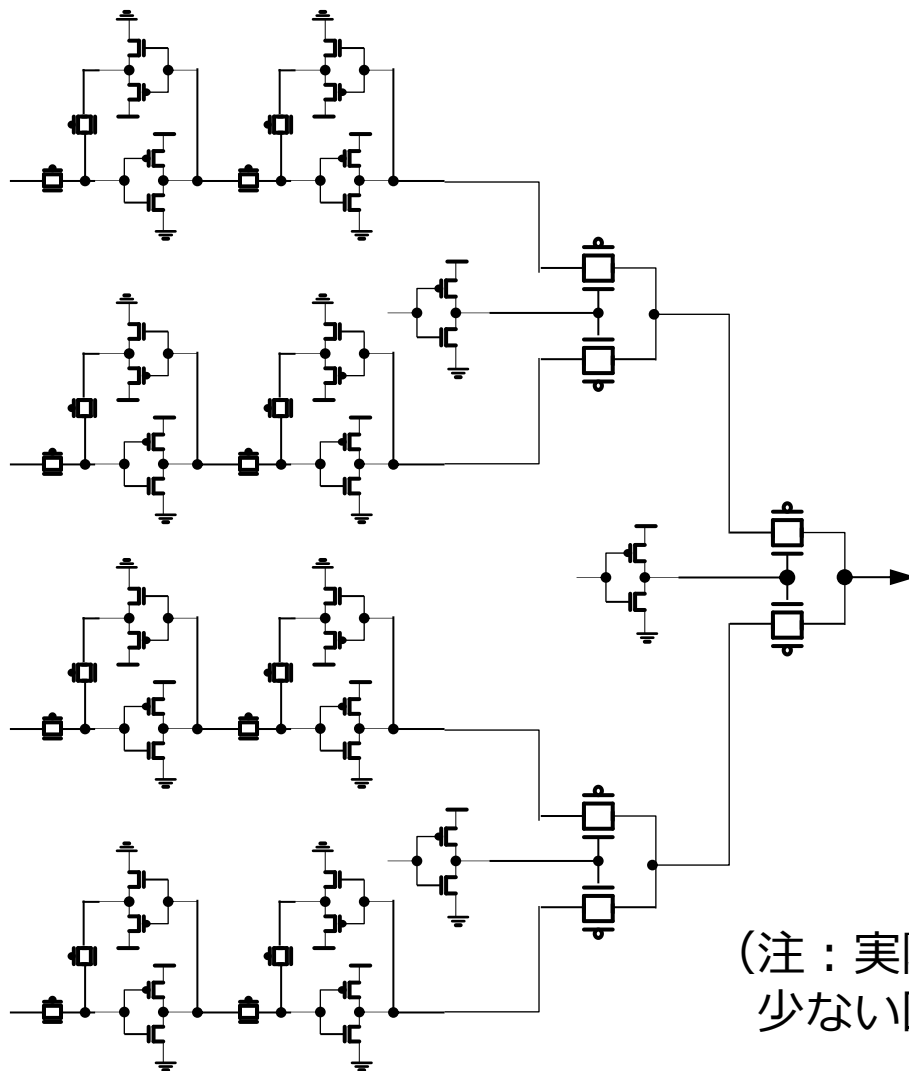


LUT vs. NAND

同じ回路を LUT で実現するのはものすごく効率が悪い

LUT : $16 \times 4 + 6 \times 3 = 82$

NAND : 4



(注 : 実際にはもう少しトランジスタ数の少ない回路でできています)

LUT で回路を構成した場合

- このほかに, LUT 間を結合するためのネットワークも必要
- 結果として, 直接回路を作るより 1 ~ 2 桁は下記の特徴が悪化
 - 回路面積
 - 遅延
 - エネルギー

CPU から FPGA にしたときに良い場合・悪い場合

■ MIPS R3000 プロセッサ :

115k トランジスタ

◇ これの上で動くプログラムを FPGA に置き換えた場合を考える



■ 1bit NAND 演算器 :

4 トランジスタ

- LUT によって 82 トランジスタになっても十分上記より小さい
(小さすぎて見えない)

■ 64bit 整数加算器 :

4k トランジスタ

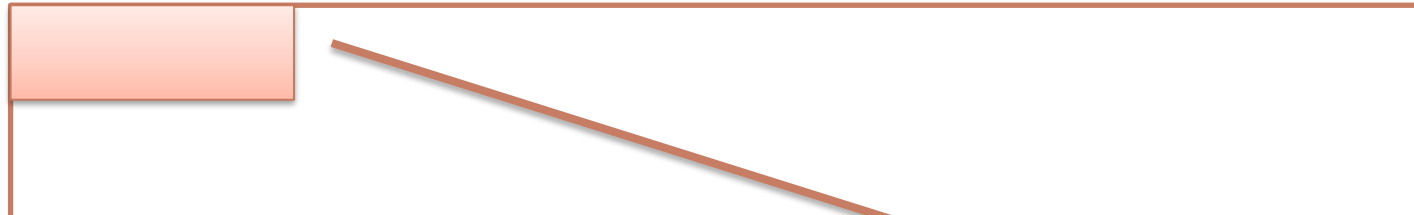
- 20倍大きくなり 80k になると, MIPS でやるのとほとんど変わらない



■ 64bit 浮動小数点 乗算+加算器 :

68k トランジスタ

- FPGA にすると巨大になりすぎるし, 何もおいしくない



FPGA の特性のまとめ

- トレードオフによって、最終的な優劣がきまる
 - FPGA 良い点：専用回路をくめば、命令制御に必要な資源が不要
 - データの受け渡のためのレジスタ・ファイルなども不要になる
 - FPGA 悪い点：回路としては一般に 1 桁から 2 桁程度性能が悪化
- 演算の種類と複雑さで FPGA 化したときにおいしいかどうかは決まる
 - 既に CPU や GPU に演算器が載っているような FP 演算などは逆効果になりかねない
- なお実際には、よく使われる回路は、FPGA では LUT ではないものが入っていることも多い
 - 加算器や乗算器などは専用の回路が入っている

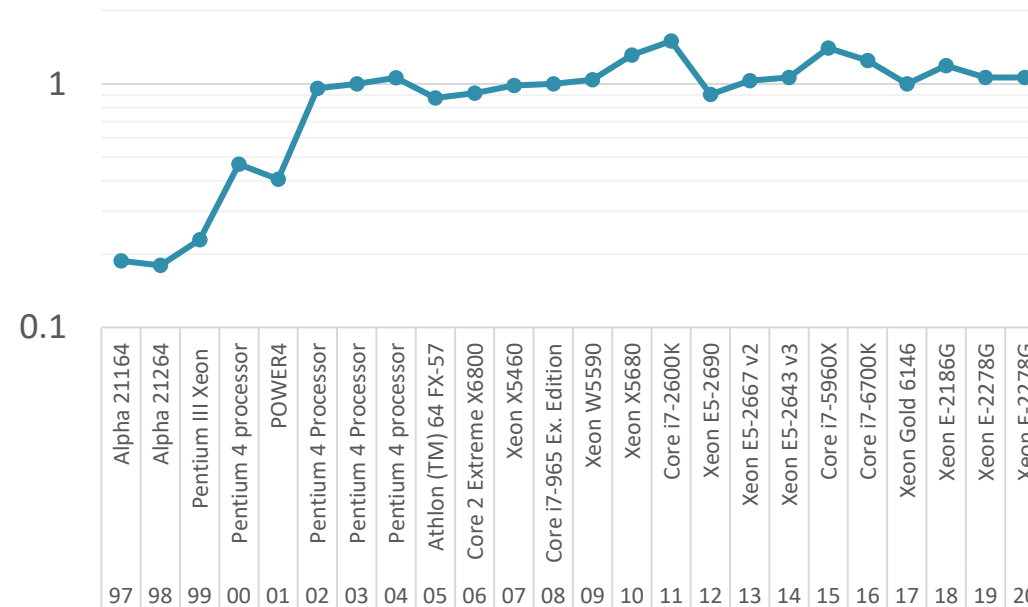
ここまでのまとめ

1. アーキテクチャの違いによる消費電力の違い
 - 回路は命令制御と演算に大きく分けられる
 - 命令制御に回路を割くと消費電力が大きくなるが、プログラマは楽に
2. FPGA による回路
 - FPGA は直接回路を作るのと比べるとかなり効率が悪い
 - CPU で動いているプログラムを FPGA の専用回路にした場合、おいしいかどうかは演算の複雑さで決まる

余談：ムーアの法則と周波数

クロック周波数

- CPU のクロック周波数 :
 - 1 秒間に何回処理を行えるかを表す
 - 性能を大きく左右
- 2002年頃からほぼ上がっていない
 - (実はここ数年はまた少し上がってきてはいる)



周波数向上がストップ

■ なぜ？ → 電圧が下げられなくなったから

- エネルギーの壁にぶつかった

1. 半導体のスケーリング
2. CPU の消費エネルギーとは何なのか
3. ダークシリコン問題

ムーアの法則

- 「半導体の集積度は3年ごとに4倍になる」
 - トランジスタのサイズを1/2に縮小（**スケーリング**）
面積： $1/2 \times 1/2 = 1/4$

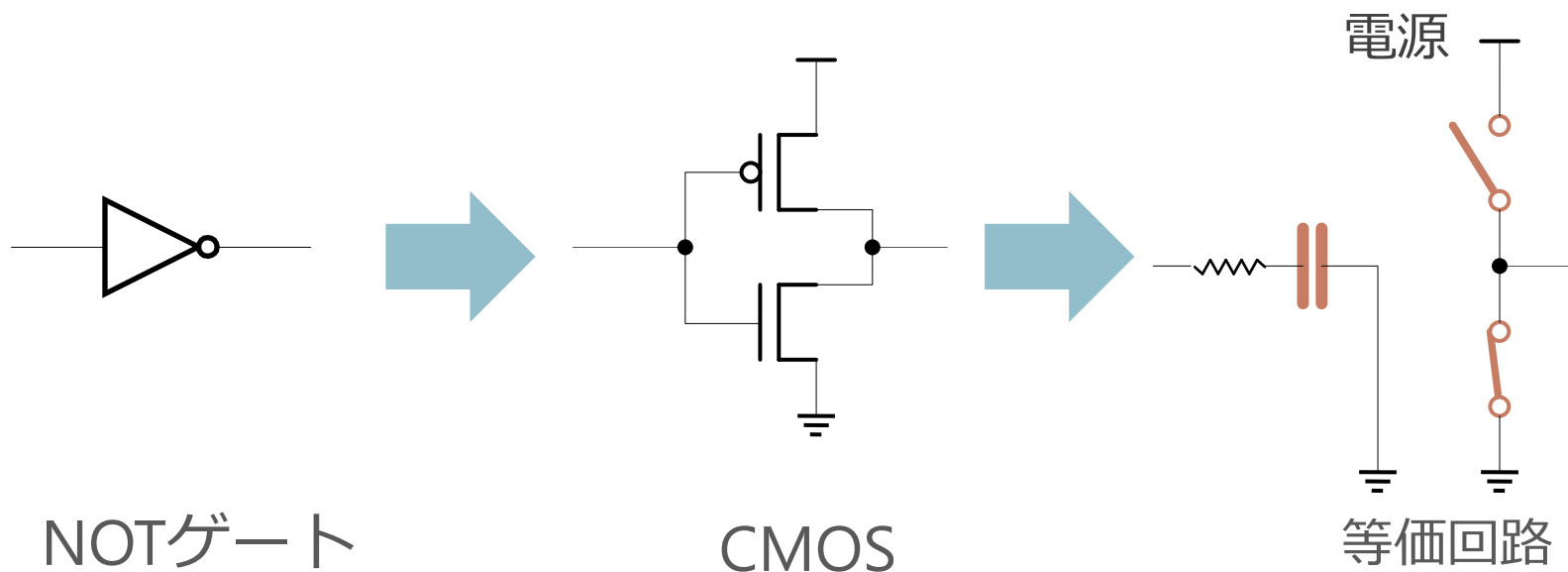


- すごいけど、数が増えるだけなの？

スケーリングの効果

- トランジスタあたりで,
 - 消費エネルギーは $1/8$ に
 - 遅延も $1/2$ に
- 等価回路を使って説明

CMOS ゲートの等価回路



- コンデンサと、連動したスイッチによって表せる
 - 充電：下のスイッチがON
 - 放電：上のスイッチがON
- CPU の計算で消費されるエネルギー：
 - スイッチ ON/OFF するためのコンデンサへの充放電

消費エネルギーと遅延

- コンデンサへの充電に必要なエネルギー
 - $\frac{1}{2} CV^2$
- サイズと電圧を $1/K$ 倍にスケーリングした場合
 - C: $1/K$ (面積 $1/K^2$, 厚さ $1/K$ より)
 - V: $1/K$
 - エネルギー: $1/K^3$
- C が小さくなり充電にかかる時間が減るので, 遅延も $1/K$ に
 - 結果として, 周波数は K 倍に
- (実際は配線などにも C が存在するので, もう少し複雑

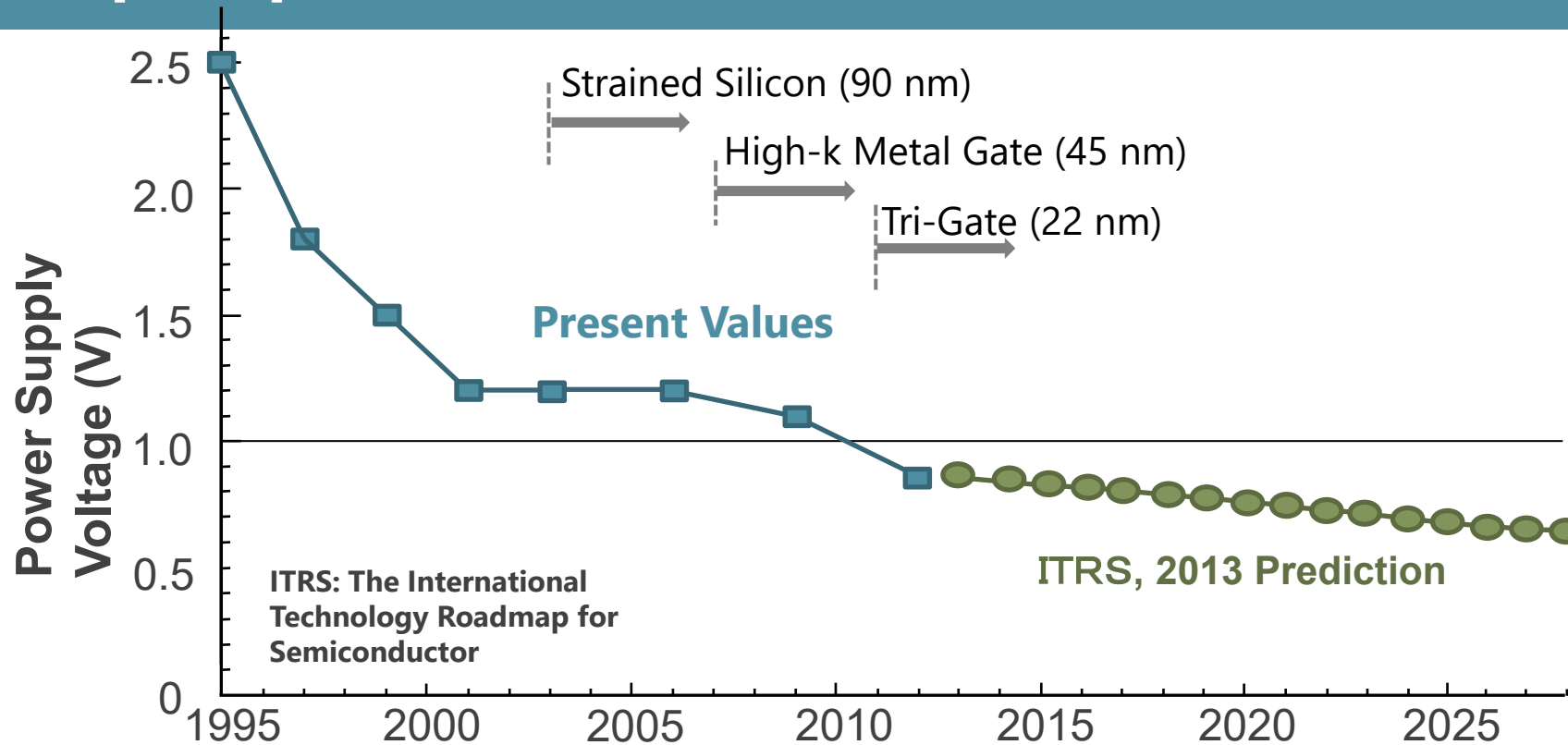
チップ全体の消費エネルギー

- $1/K$ 倍にスケーリングした場合, チップ全体では,
 - 個々のトランジスタのエネルギー : $1/K^3$
 - トランジスタの個数 : K^2
 - 周波数 (動作回数) : K
 - 全体の消費エネルギー : 1

- まとめると, スケーリングによって
 - 同じ大きさのチップに搭載できる回路量が増えるのはもちろん,
 - 消費エネルギーは一定のまま,
 - 周波数もすごい向上!

ところが、電圧が下げられなくなった...

グラフは [Sato17] より



- 2000 年過ぎから、電圧低下がほぼとまった
 - ◇ 電圧が下がりすぎて、ON / OFF の境界の閾値が 0V 付近に
 - ◇ OFF にしたいときも、閾値との距離がとれない
 - ◇ 微妙に ON になり電流が漏れてしまう（リーク電流という）

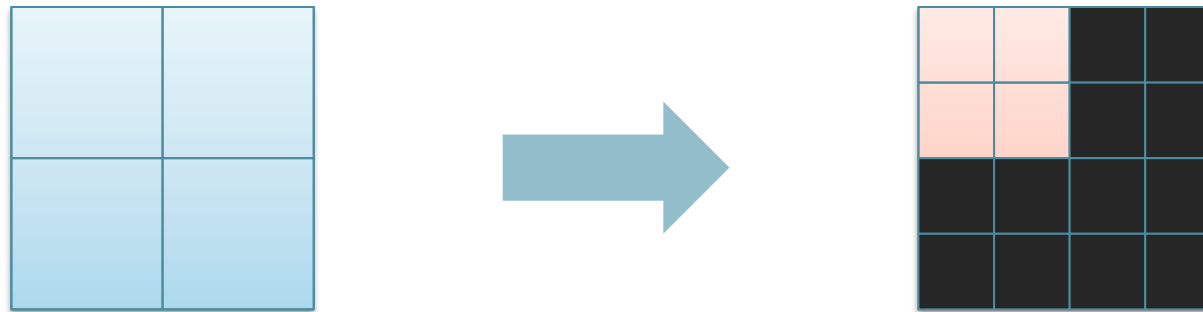
電圧が下がらないとどうなるのか

- $1/K$ 倍にスケーリングした場合
 - 回路面積: $1/K^2 \rightarrow 1/K^2$
- トランジスタ 1 個の 1 回のスイッチにかかるエネルギー
 - C: $1/K \rightarrow 1/K$
 - V: $1/K \rightarrow 1$
 - エネルギー: $1/K^3 \rightarrow 1/K$ ($E = \frac{1}{2}CV^2$ より)
- トランジスタ 1 個の単位時間あたりのエネルギー
 - スイッチ 1 回: $1/K^3 \rightarrow 1/K$
 - 周波数 (動作回数): $K \rightarrow K$
 - エネルギー: $1/K^2 \rightarrow 1$
- トランジスタは小さくなったのに, エネルギーが減っていない!

行き着く先：ダークシリコン

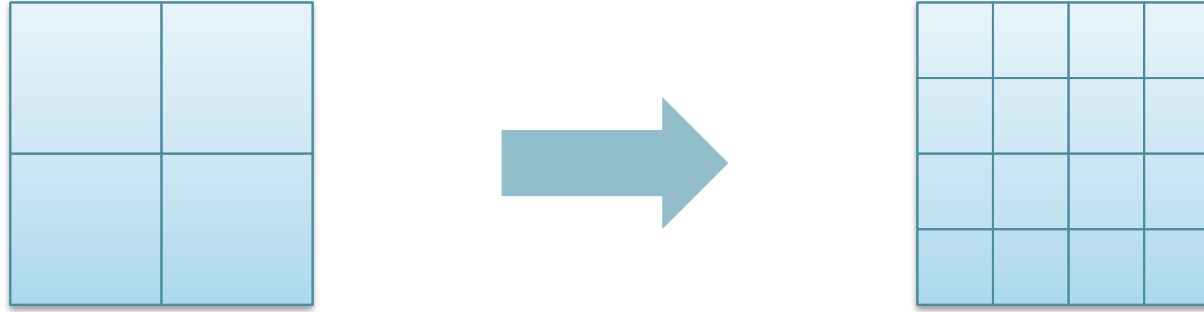
[Goulding10, Esmaeilzadeh12]

- トランジスタは小さくなったのに，エネルギーが減らない！
 - サイズだけ小さくして，電圧を下げていないから
- 電力密度があがってしまう
 - スケール前のチップ全体と，スケール後の赤い部分は同じエネルギーを消費



- チップへの電力供給はもう限界で，これ以上増やせない
 - 電源ピンに流せる電流量や冷却能力の限界
 - 使えない（ダーク）シリコンが...

とはいえ、いまのところ



- まだ、ダークにまでは至っていない
 - デバイス技術が進歩
 - 電圧が一応ちろちろ下がってはいる
 - 周波数を上げなくした
 - 動作回数が減るので、電力にダイレクトに効く
 - あまりスイッチ（充放電）しないキャッシュなどを多めに入れた
- まとめ：電圧が下げられなくなったので、電力密度増大をおさえるために周波数を上げるのをやめた

ここまでのまとめ

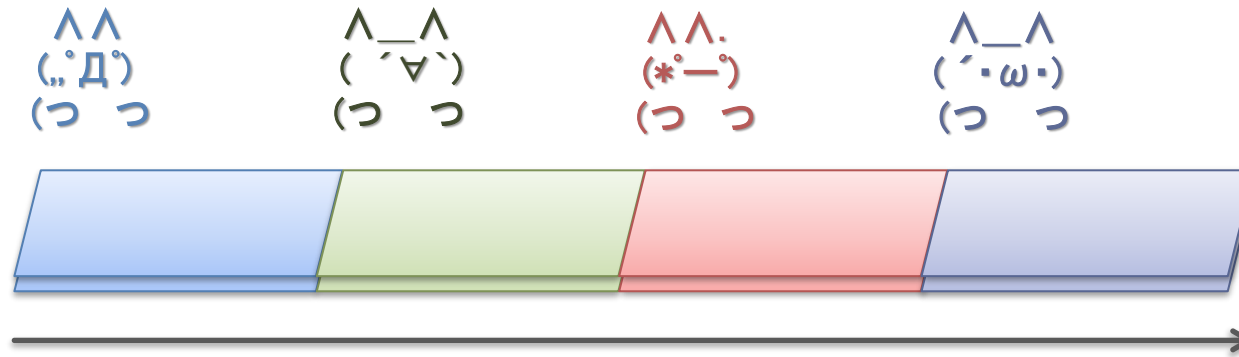
- 回路の消費エネルギー
 1. アーキテクチャの違いによる消費電力の違い
 2. FPGA による回路
 3. (余談) ムーアの法則と周波数

命令パイプライン

命令パイプライン

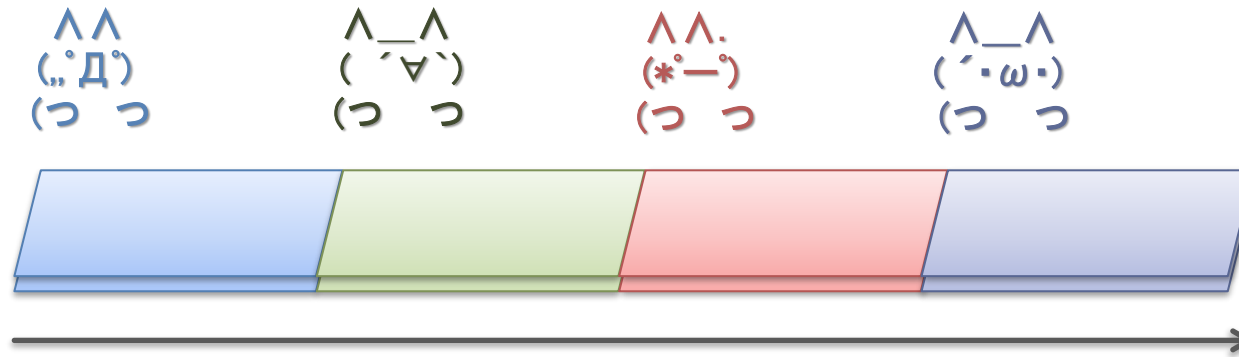
1. シングル・サイクル・プロセッサの動作
 - パイプライン化を前提とした構造のものを使って復習
 - 全ての命令の処理が 1 サイクルで完結
2. 上記のパイプライン化
 1. 具体的にどうパイプライン化するか
3. (次回以降) ハザード

導入：工場のラインを考える



- ベルトコンベアのラインの上を製品が流れていく
 - 4 人の人が、それぞれの工程の作業をおこなって完成
- 上のように1つしか製品をながさないで、
 - 各人は他の人が作業している間はヒマ

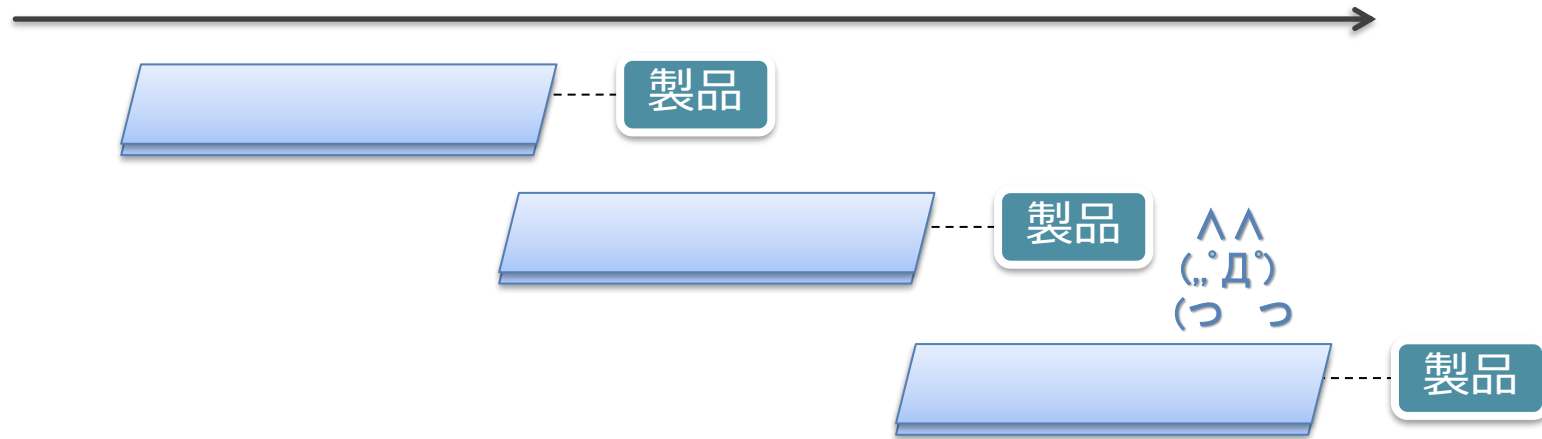
導入：工場のラインを考える



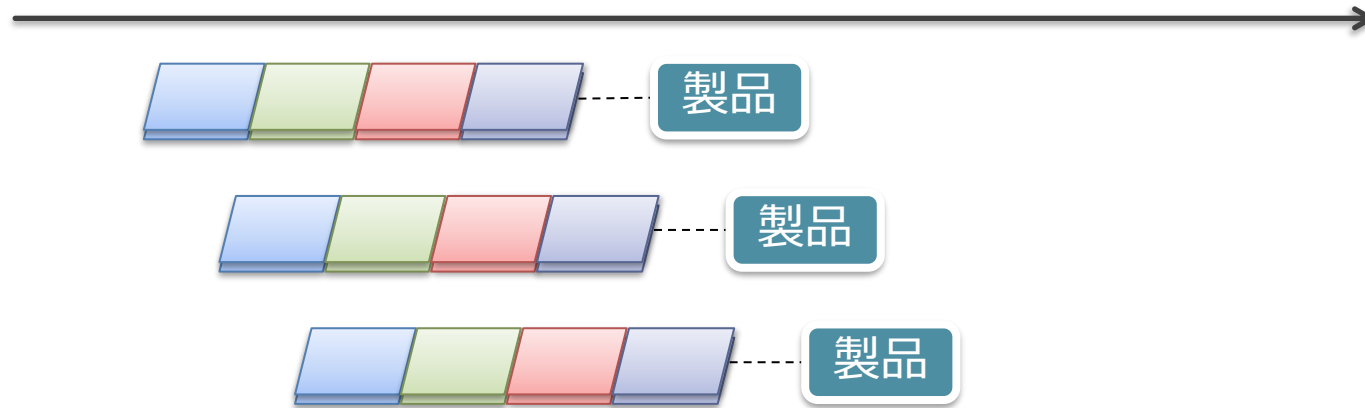
- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
 - さっきの4倍の速度で製品ができあがっていく
- これが 命令パイプライン

パイプライン化による性能向上

パイプライン化しない場合



パイプライン化した場合



シングル・サイクル・プロセッサの動作

1. シングル・サイクル・プロセッサの動作

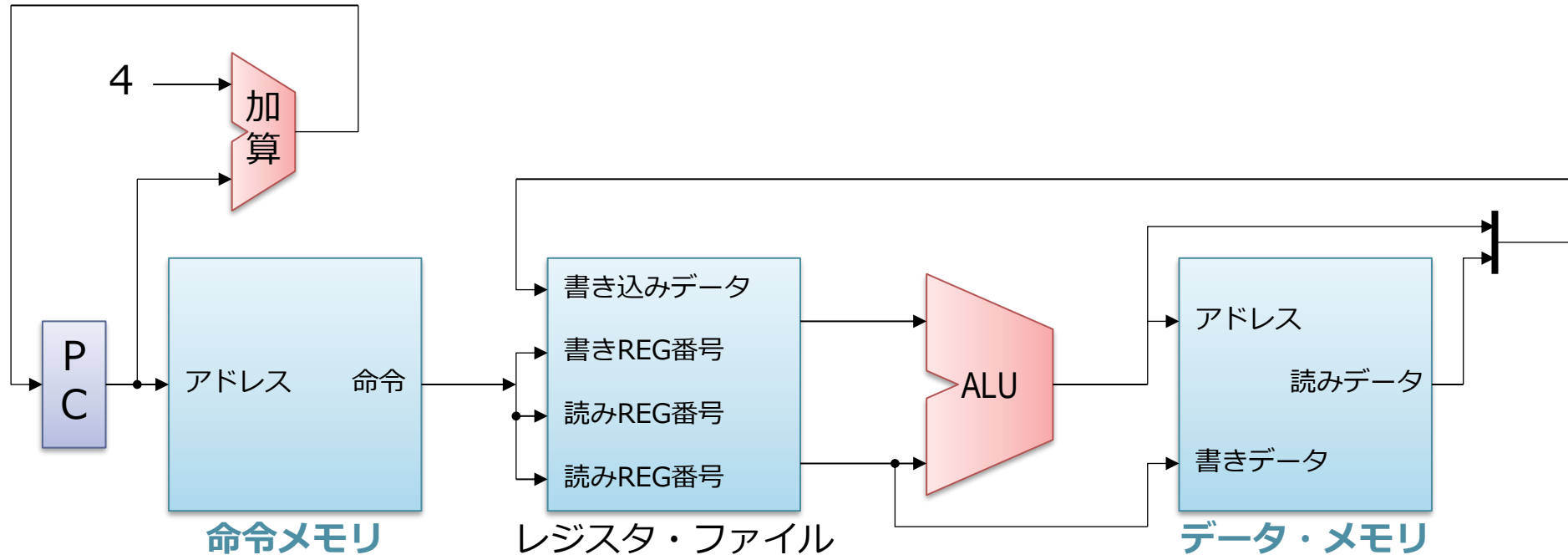
- パイプライン化を前提とした構造のものを使って復習
- 全ての命令の処理が 1 サイクルで完結

2. 上記のパイプライン化

1. 具体的にどうパイプライン化するか

3. (次回以降) ハザード

ベースとなるシングル・サイクル・プロセッサ



■ 以前説明したものとの違い：

- メモリが命令メモリとデータメモリに別れている
- 算術 & 論理演算，ロード，ストアのみを実行可能
 - 分岐とジャンプは，簡単のために今は考えない

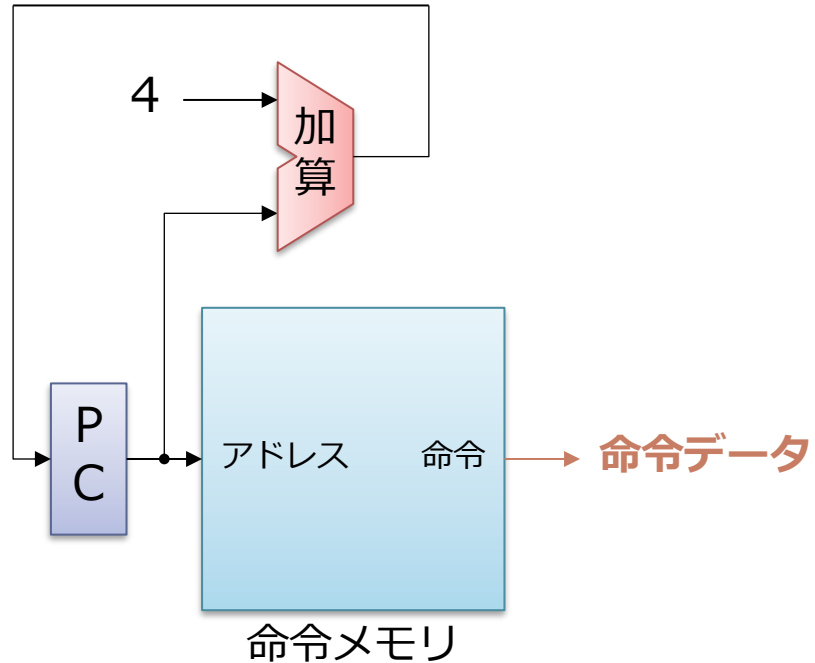
命令の実行フェーズ

■ 実行フェーズ

1. フェッチ
2. デコード
3. レジスタ読み出し
4. 実行
5. レジスタ書き戻し

■ RISC-V の加算命令を実行する流れをざっとみる

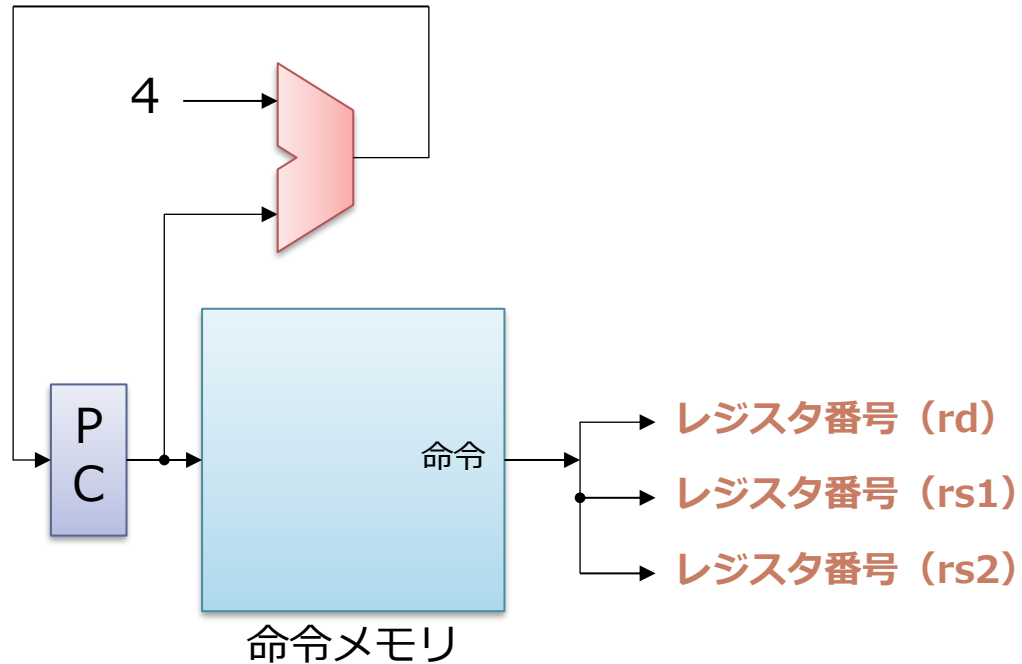
命令フェッチ



■ 命令メモリから命令を読み出す

- 命令メモリを順に読んでいくため、PC は毎サイクル加算される
- 足している 4 は、RSIC-V では命令の幅が 4 バイトだから
- 基本的に、この部分はどの命令でも変わらない

命令デコード

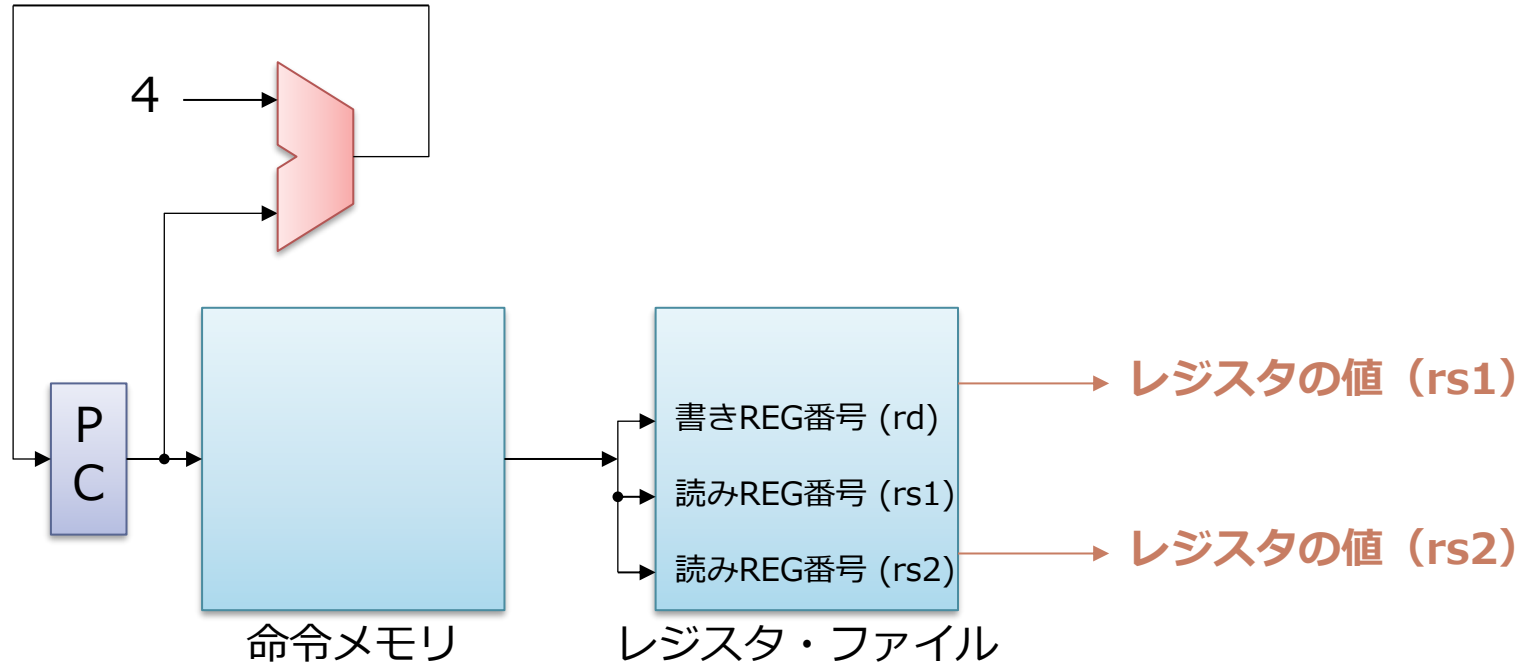


ADD : $x[rd] \leftarrow x[rs1] + x[rs2]$



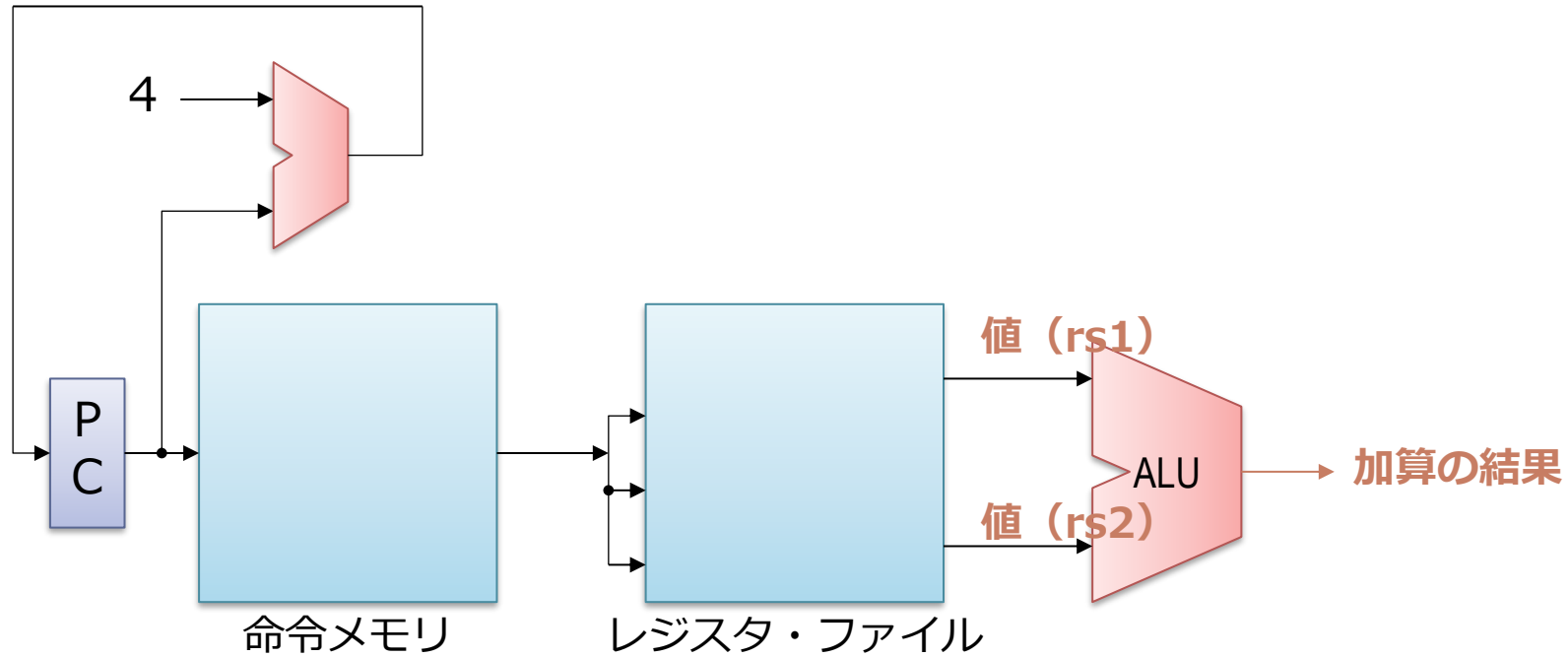
- 取り出した命令からレジスタ番号を表す部分のビットを取り出す
 - ソース (rs1, rs2) とディスティネーション (rd)

レジスタ読み出し



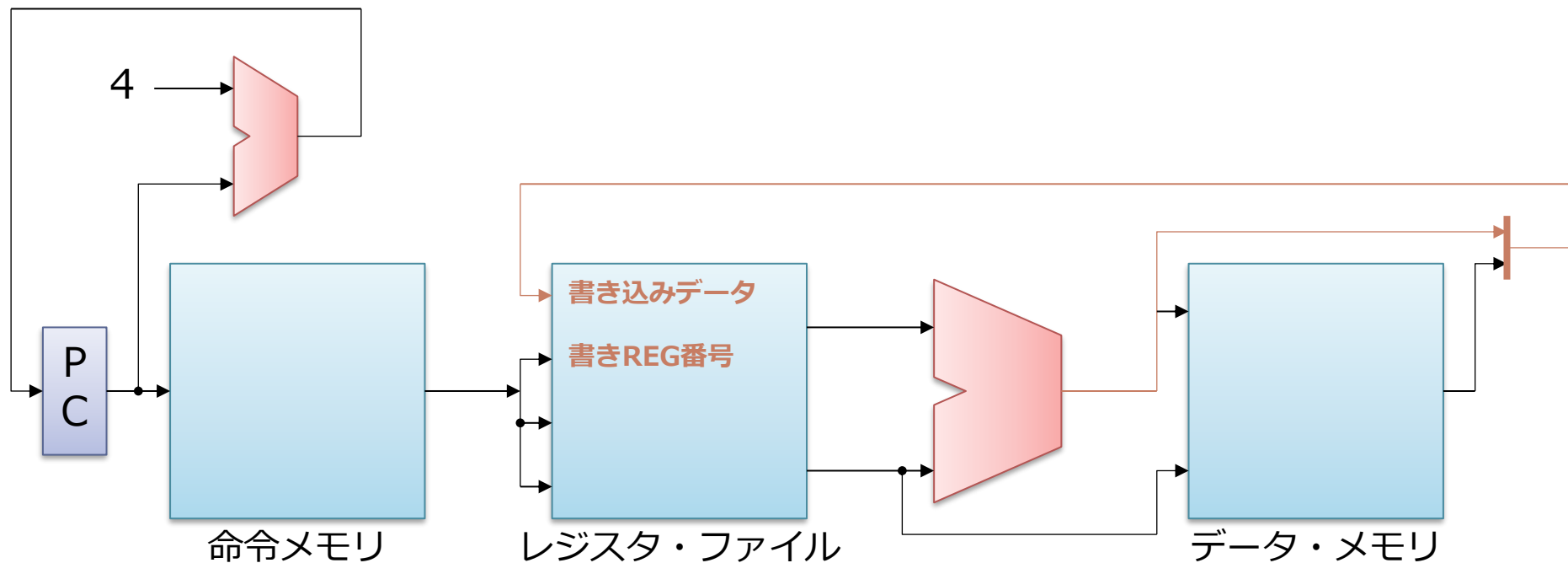
- デコードで得られたレジスタ番号を使って RF にアクセス
 - ソース・オペランドの値を読み出す

実行



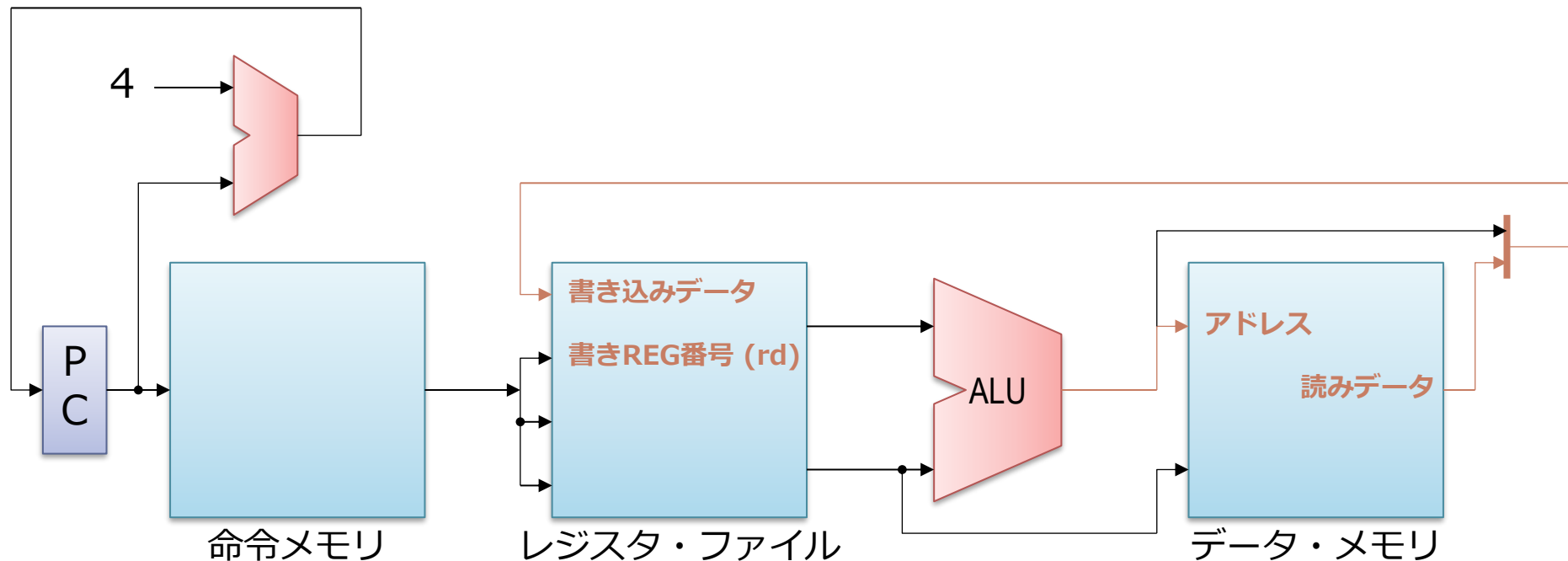
- RF から読みだした 2 つの値を加算

レジスタ書き戻し



- 加算の結果をレジスタ・ファイルに書き戻す
 - データ・メモリには用がないので何もしない

ロードの場合：メモリ・アクセスが加わる



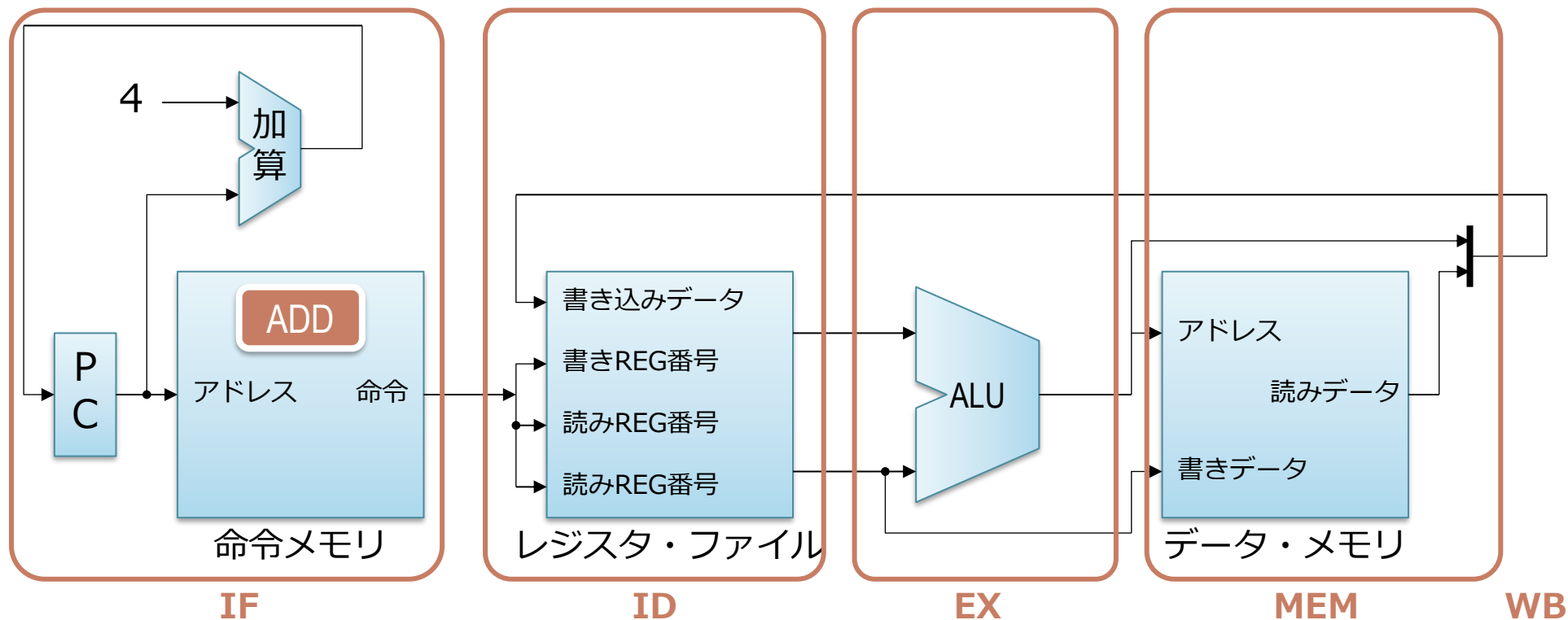
LW : $x[rd] \leftarrow (x[rs1] + \text{immediate})$



■ 加算命令との違い：

- アドレスの計算 ($x[rs1] + \text{immediate}$) を ALU でやる
- 得られたアドレスでデータ・メモリにアクセス

各処理は基本的には左から右に流れる



- 特定のユニットで仕事をしている間，他の部分は遊んでいる
- パイプライン化
 - これをもとに，導入で話したように処理をオーバーラップさせる

パイプライン化

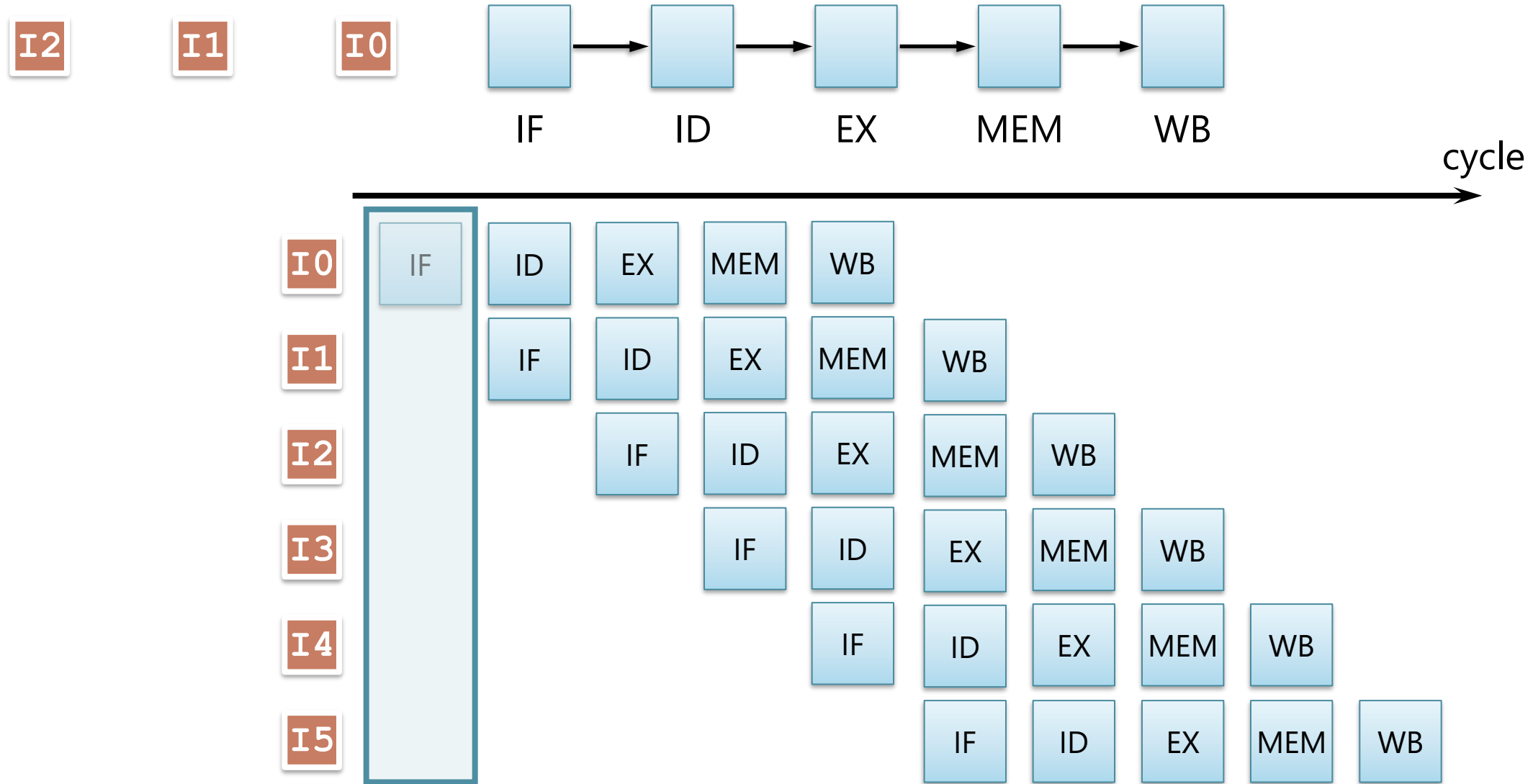
もくじ

1. シングル・サイクル・プロセッサの動作
 - パイプライン化を前提とした構造のものを使って復習
 - 全ての命令の処理が 1 サイクルで完結
2. **上記のパイプライン化**
 1. 具体的にどうパイプライン化するか
3. (次回以降) ハザード

パイプライン化

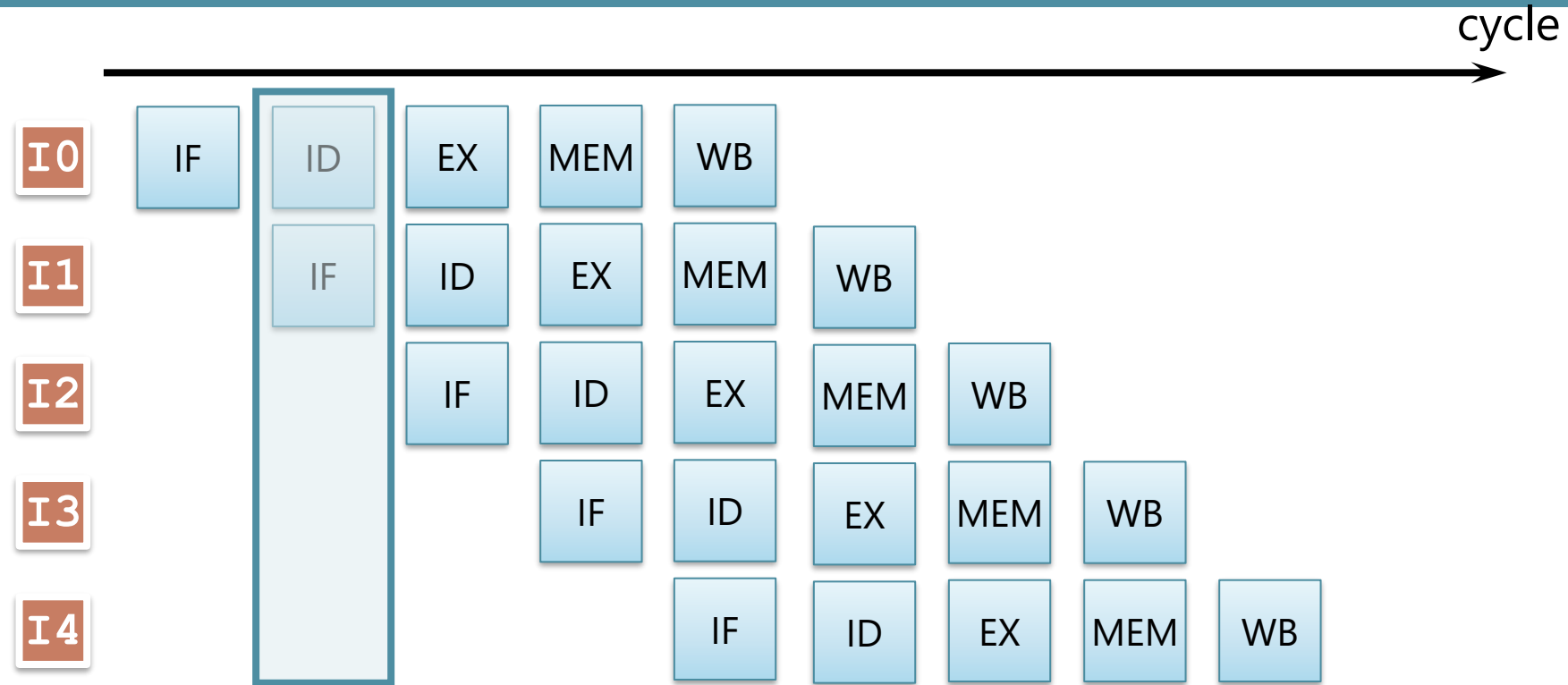
- 回路のまとまりをオーバラップさせる単位にする
 - この単位をステージと呼ぶ
- ステージ
 1. IF : 命令フェッチ
 2. ID : デコードとレジスタ読み出し
 3. EX : 実行
 4. MEM : メモリ・アクセス
 5. WB : レジスタ書き込み

命令パイプラインの実行の様子



パイプライン・チャートの見方

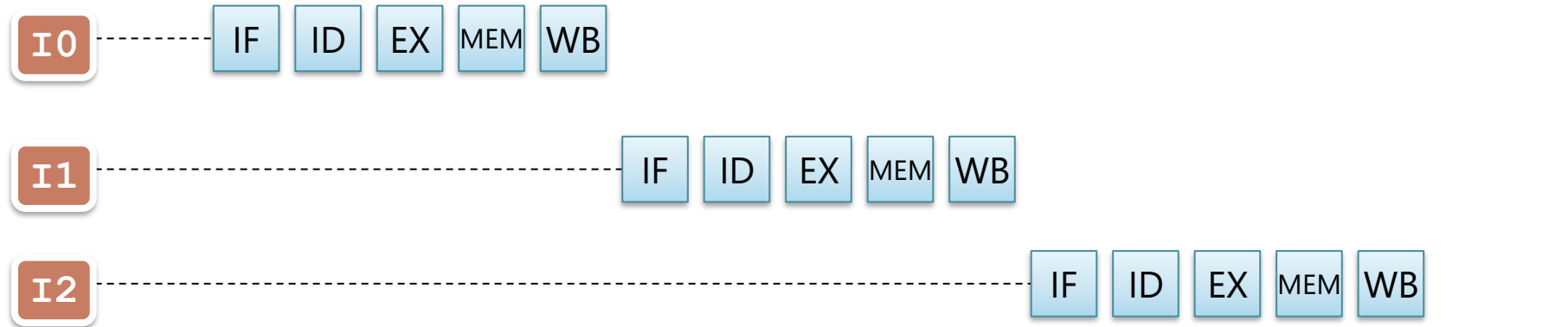
ここから先で多用されるので重要



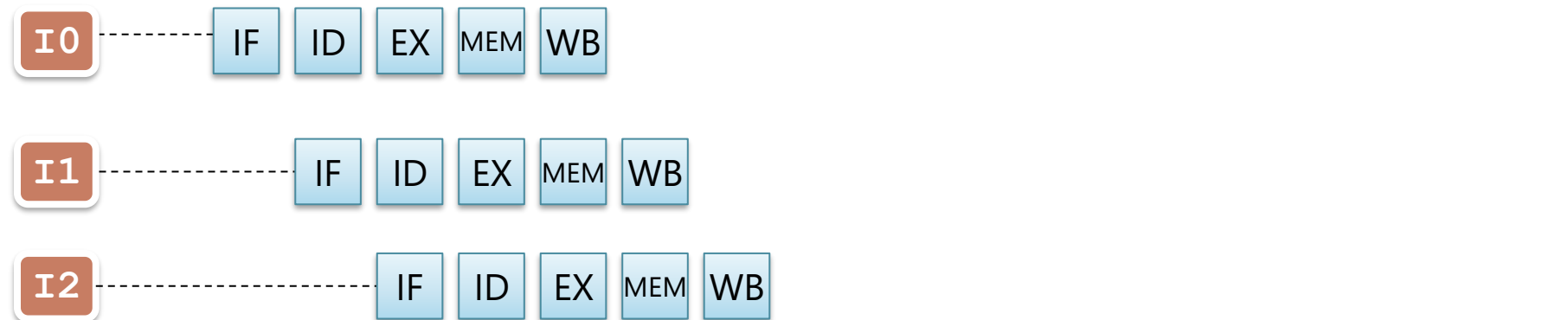
- 左から右にむかって時間は進む
- 上から下にむかって命令が実行順に置かれる
- 各ステージを表す四角は左側にある命令がその時そこにいることを示す
 - 上記では2サイクル目に, I0 が ID に, I1 が IF で処理されている

パイプライン化による性能（スループット）向上

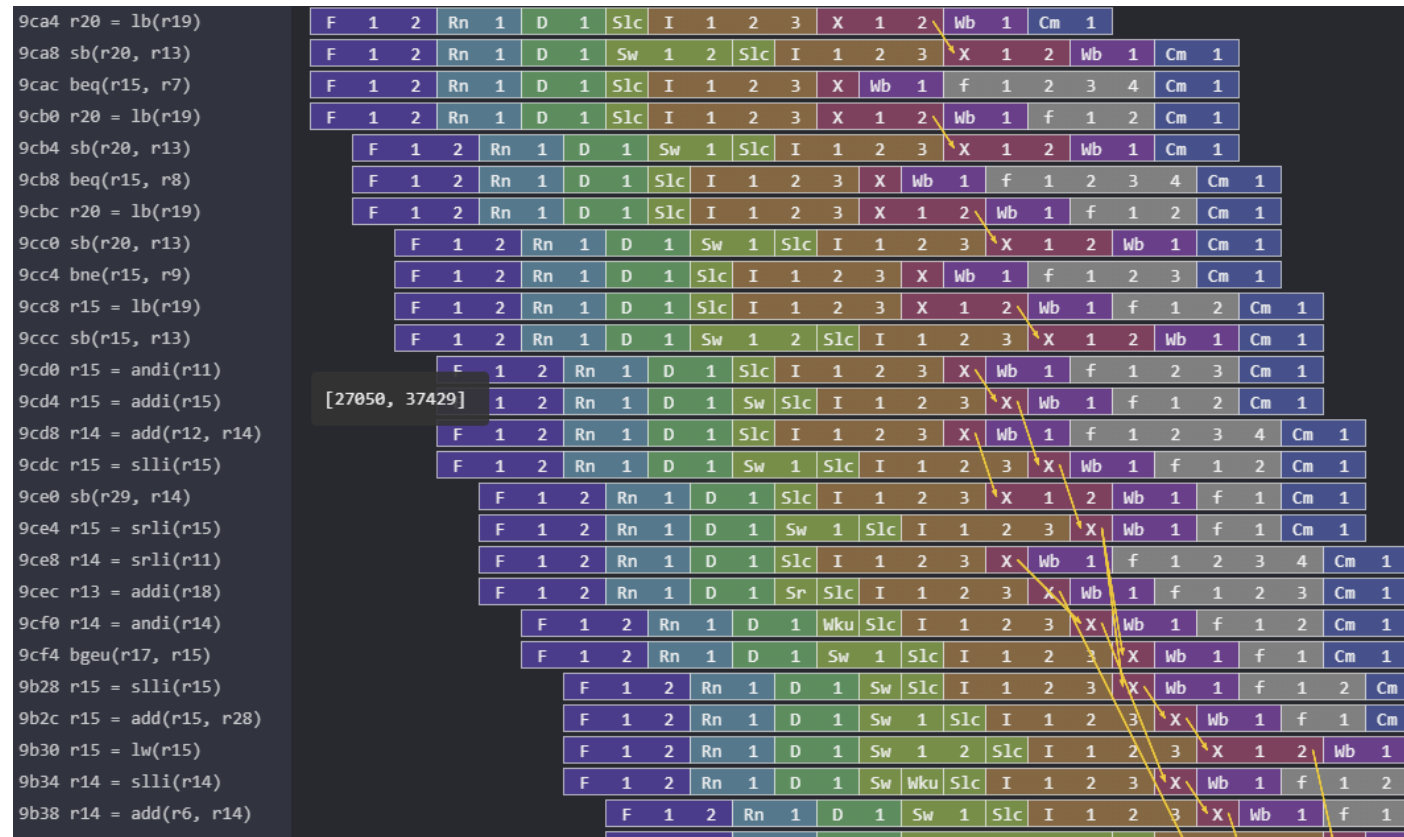
パイプライン化しない場合



パイプライン化した場合



余談：実際の CPU を実行した場合のパイプライン



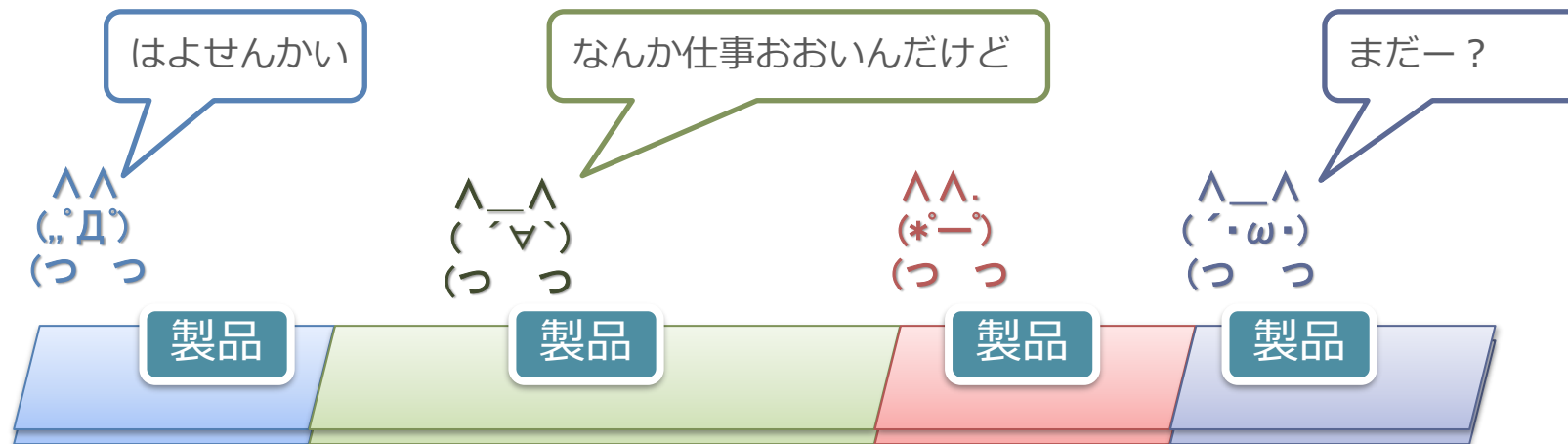
- 塩谷が開発している RISC-V CPU（RSD）の実行を可視化したもの
 - <https://github.com/rsd-devel/rsd>
 - out-of-order 実行をしているので，途中からプログラム順とは異なるタイミングで実行が進んでいる

パイプライン化の効果

- レイテンシ (latency) : 短くならない (か, やや延びる)
 - 一続きの処理が始まってから終わるまでにかかる時間
 - この場合, 1命令の始まりから終わりまでの処理時間
 - 原理的に短くならない (ステージ間にFF が入る分のびる)
- スループット (throughput) : ステージ数倍だけ上がる
 - 単位時間当たりの処理量
 - この場合, 単位時間あたりに実行される命令数

ステージを「どこで」切るか

- 大きな回路のまとまりをステージにする
 - 回路のまとまりが大きい → 遅延も大きい
- この遅延の大きさが揃っていないと、綺麗にうごかない
 - パイプライン全体は、一番遅いステージの遅延にあわせて動く
 - 他の人が仕事が終わったからと言って、先に送れない
- 良くない例：緑の人だけ仕事が多いので、全体が動かせない



ステージを「どこで」切るか

■ ステージ

1. IF : 命令フェッチ
2. ID : デコードとレジスタ読み出し
3. EX : 実行
4. MEM : メモリ・アクセス
5. WB : レジスタ書き込み

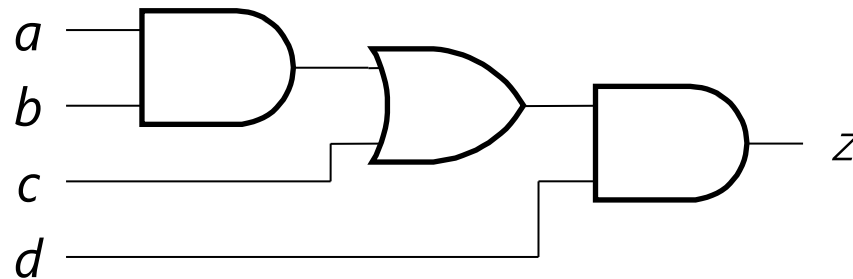
■ 上記では、デコードとレジスタ読み出しが ID ステージにまとめられている

- デコードにかかる遅延はほとんどない
- 読み出した命令からオペランドを取り出すのは、単に信号線を繋ぐだけで良い

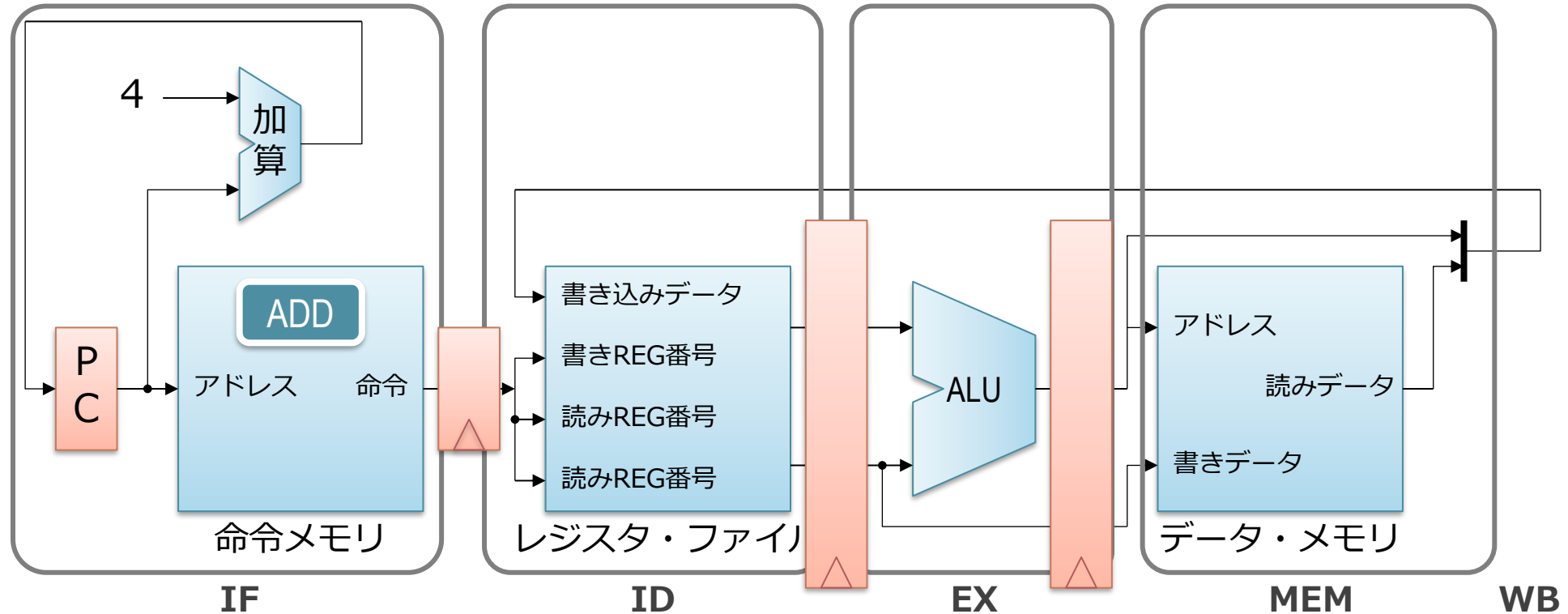
ステージを「どうやって」切るか

- シングル・サイクル・プロセッサの回路に適当に間隔をあけて命令を流せばよいというものもない

1. 各ステージを完全に同じ長さにするのは凄く難しい
 - 同じ長さ=同じ遅延=全く同じ段数の組み合わせ回路
2. 長いステージであっても信号は絶えず変化する可能性がある
 - 短いパスから順に出力に反映される
 - たとえば下の回路で a, b, c, d が全て変化したとすると、まず d の変化が z に反映し、次に c が...



パイプライン化（オーバーラップ）の実現方法



- 各ステージの間に, D-FF (オレンジの四角) を入れる
 - WB の書き込みについては, レジスタ・ファイル自体がクロックに同期して書き込みが行われるので D-FF は不要
- 各ステージの処理が早く終わっても, 次のクロックまでは D-FF で信号の伝搬は止まる

余談：非同期回路やウェーブ・パイプライン

- クロックによる同期化を使わずにパイプラインを作る方法もあるにはある
- やり方：
 1. 色々な方法でステージ間の遅延の大きさを気合いで揃える
 2. 一定間隔でデータを流す
- 設計 & 動作させることがすごく難しいので、主流ではない
 - 特に、高速動作がかなり難しい

出欠と感想

- 本日の講義でよくわかったところ，わからなかったところ，質問，感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので，そこをお願いします
 - パスワード : arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは来週の講義開始までに設定してあります
 - 仕様上「遅刻」表示になりますが，特に減点等しません
 - 来週の講義開始までは感想や質問などを受け付けます

- 以下のスプレッドシートの自分の席のところに学籍番号を書いてください
 - https://docs.google.com/spreadsheets/d/1Xte7HdUVfoAGclTWzx-sBQG_ObaNQO3kbH5fV5HD7oo/edit?usp=drive_link
 - 学生側から見た場合の配置で（画面上が教団側）、席の対応する行と列のところに書いてください

質問や感想

- 完備性の話は学部の実験で一回言及されていたけどあんまり納得しなかったのですが、一回自分で示してみたいですね。CMOSを水流システムに例える部分は楽しかったです。だいぶ分かりやすく直感的に理解できました。

- CMOSの水流への例えは非常に分かりやすいですね。D-FFがメモリのよ
うな動きをすることは習いましたが、その具体的な動作は今回初めて習
いました。だからFLIP FLOPですね。

- マインクラフトでプロセッサを作る動画を見てこの分野を志した面もあるので、言及してくださって嬉しかった。
- Minecraftがはじめて論理回路に触れるきっかけだったので講義で紹介されて懐かしい気分になりました。

- CMOSじゃない汎用プロセッサを作ってみたくなりました、水流とか、機械構造（ラッチ）とか、リレーとか、

- 消費電力は設計する上で大切な要素ですが、論理合成結果のみだけの電力だと実際に動かした時の消費電力とズレることが多いので、そこがなんとかなると嬉しいとずっと思っています。(チップが動作しないことがザラにあるので)

- とくに面白かったのは、水流を用いた回路の説明です。これまでも電気回路を水流にたとえる説明を見たことはありますが、今回の講義ではその比喻がかなり細かいところまで対応していて、とても納得感がありました。単に「電流は水の流れのようなもの」と説明するだけでなく、タンクに水が溜まることや、栓の開閉、パイプに水が残ることなどを使って、CMOS のスイッチングや消費エネルギーの違いまで説明していた点が特に分かりやすかったです。CPU が電力を消費する理由を、抽象的な式だけでなく、物理的なイメージとして理解できたのは大きな収穫でした。

- 半導体が微細化すると1素子あたりの電気量が減るかと思ったのですが、これは誤解でしょうか?また、消費電力が増えているのは素子あたりの電気量は減るが素子をおける面積が増えるために消費電力が高まっていると思ったのですがこの理解はただしいでしょうか?

- ・ nandゲートができればコンピュータが作れる (今日何度か主張されていたこと)
 - ・ nandゲートができればチューリング完全なシステムが構築可能
- この辺りの主張について、厳密な用語の指す意味も含めて復習しようと思った
- (どちらも正しいと思っているが文脈が違うのでは？とも感じている)

- そういえばマインクラフトでも各種ゲートの概念を作れる投稿を最近見かけましたね

- LLMが普及して以来、Linux Kernel へのコントリビュートが大幅に増えているように、RISC-V界隈も設計レベルではより活発になるのではと思いました。

- CMOSの水流を用いた説明が非常にわかりやすかったです。ここ一年ほどでキオクシアの株が大きく伸びた要因の一つにNANDが評価されたことがあり、「NANDごときでそんな評価されるんだ」と思っていました。NANDがコンピュータの礎になる最重要な論理ゲートであることが分かり少し合点しました。

- 5/7（木）の休講による講義スケジュールの変更があるか、または自習やオンデマンドで補われるか伺いたいです。

- クロックの配線の話について、確かに同期クロックって大変だなと理解しました。自分が使っている100万PEを1チップに搭載したアーキテクチャでは、各PEが independent clock を持っていることも納得です。
(極論、クロック周波数が同じであれば問題ないと思うので)

- CMOS半導体などの原理を、水が流れる様子などを用いて説明して頂いたのは非常に分かりやすかったです。最後の方で紹介していた半導体デバイスなどの効率的な3D配置方法などは、具体的にどのようなものがあるのですか。